

Tutorial - Semana 3, Encontro 1

Introdução

Nas Semanas 1 e 2, o Vertical Slice ganhou uma Scene organizada em Nodes (`level_exploration.tscn`) e um Player (CharacterBody3D) que se move pelo nível através de um Input Map próprio. Até aqui, porém, o nível é apenas um `Chao` genérico, sem superfície definida e sem relevo. Este encontro resolve o próximo problema universal de qualquer engine 3D: como definir a aparência de uma superfície e como modelar grandes extensões de terreno sem esculpir manualmente cada polígono? O primeiro problema é resolvido por um **Material Graph** — uma representação nodal de shader —, e o segundo por uma ferramenta dedicada de escultura de terreno, o addon **Terrain3D**, que não existe nativamente no Godot.

Este tutorial dá continuidade direta à Semana 2 — o Player e a Scene `level_exploration.tscn` já devem existir e serão reabertos, não recriados.

Objetivos da semana

- Explicar o Material Graph como implementação nodal de um shader.
- Explicar o papel do addon Terrain3D como ferramenta de terreno sem equivalente nativo no Godot.
- Criar um material simples e modelar um terreno básico para o nível do Vertical Slice.

Resultado esperado ao final da semana

Ao final da Semana 3 (Encontros 1 e 2), cada estudante terá um nível de teste com terreno, material e iluminação global ativa, exportado como o primeiro build executável do Vertical Slice — encerrando o Módulo 1. Este tutorial cobre apenas o **Encontro 1**: a aplicação de material e a modelagem do terreno, ainda sem iluminação global nem exportação.

Pré-requisitos

- Player (CharacterBody3D) controlável na Scene `level_exploration.tscn`, com Input Map configurado (ver Tutorial - Semana 2, Encontro 1 e Encontro 2).
 - Addon Orchestrator instalado e habilitado no projeto.
-

Antes de começar

O que o estudante deverá possuir antes desta semana

- O projeto Godot da Semana 2, com o Player controlável nas quatro direções sobre o `Chao` da Scene `level_exploration.tscn`.

Arquivos necessários

- Addon **Terrain3D** instalado no projeto (via AssetLib do Godot ou download manual do repositório oficial) e habilitado em **Project > Project Settings > Plugins**.

Assets utilizados

- **Kenney Prototype Kit** (CC0), conforme PROJECT_ARCHITECTURE.md (seção 3), usado para dar textura de referência ao material do graybox. É o primeiro pacote Kenney utilizado na disciplina.

Projeto esperado

- Projeto aberto no Godot 4.7, com a Scene `level_exploration.tscn` pronta para receber material e terreno.
- Addon Terrain3D já instalado e habilitado em **Project Settings > Plugins**.
- Pacote Kenney Prototype Kit importado em `assets/` (subpasta a ser criada, se ainda não existir).

Imagem sugerida

Objetivo: mostrar a aba **Plugins** do Project Settings com o Terrain3D habilitado.

Enquadramento: captura de tela da janela Project Settings, aba Plugins. Elementos

importantes: linha do addon "Terrain3D" com o status marcado como "Active". Destaque: a

caixa de seleção que ativa o plugin. Legenda sugerida: "Addon Terrain3D habilitado no projeto antes de iniciar a escultura do terreno."

Parte 1 — Material Graph como implementação nodal de um shader

Objetivo

Entender como uma engine define a aparência de uma superfície sem exigir código de shader escrito à mão, antes de aplicar qualquer material no editor.

Conceito

Toda superfície visível em um jogo 3D precisa responder à mesma pergunta: como ela reage à luz? Cor, brilho, rugosidade e textura combinam-se para produzir essa resposta. Escrever um shader manualmente para cada superfície do jogo seria inviável para a maioria das equipes — por isso, engines modernas oferecem uma representação nodal desse problema, o **Material Graph**: um grafo de nós que combina texturas, cores e parâmetros visualmente, sem exigir código de shader direto na maioria dos casos.

No Godot, essa solução tem duas camadas. A mais simples é o **StandardMaterial3D**, um material pronto com parâmetros expostos diretamente no Inspector (Albedo, Roughness, Metallic, entre outros) — suficiente para a maior parte das superfícies de um projeto como o desta disciplina. Quando o **StandardMaterial3D** não é suficiente, o Godot expõe o **Visual Shader**, seu grafo nodal completo, equivalente conceitual ao Material Graph de outras engines. Este encontro trabalha exclusivamente com **StandardMaterial3D**, adequado ao graybox do Vertical Slice.

Passo a passo

1. Reabra o projeto do Vertical Slice e, no FileSystem Dock, dê duplo clique em `scenes/levels/exploration/level_exploration.tscn` para reabrir a Scene das Semanas 1 e 2.
2. Confirme, no FileSystem Dock, que o pacote Kenney Prototype Kit foi importado em `assets/` (crie a subpasta `assets/prototype/` caso ainda não exista, seguindo a organização do `PROJECT_ARCHITECTURE.md`, seção 8).
3. Crie a pasta `materials/` na raiz do projeto, caso ainda não exista.
4. No painel Scene, selecione o Node `Chao`.
5. No Inspector, localize a propriedade **Material Override** (ou **Surface Material Override**, dependendo do tipo de `MeshInstance3D` usado) e clique em **New StandardMaterial3D**.
6. Clique no material recém-criado para abri-lo no Inspector e salve-o como recurso externo em `materials/chao_prototype.tres` (botão direito sobre o material > **Save As**).

7. No Inspector do material, expanda a seção **Albedo** e atribua uma textura do Kenney Prototype Kit (por exemplo, uma textura de grade ou concreto) ao campo **Texture**.
8. Ajuste o parâmetro **Roughness** para um valor entre 0.6 e 0.8, produzindo uma superfície fosca compatível com um piso de teste.
9. Confirme visualmente, no Viewport, que a textura aparece corretamente esticada sobre o `Chao`, sem distorção exagerada.
10. Salve a Scene (**Ctrl+S**).

Resultado esperado

O Node `Chao` possui um `StandardMaterial3D` próprio, salvo como recurso externo em `materials/chao_prototype.tres`, com uma textura do Kenney Prototype Kit aplicada ao Albedo e um valor de Roughness ajustado.

Verificando

1. No FileSystem Dock, confirme que o arquivo `materials/chao_prototype.tres` existe.
2. Selecione o `Chao` e confirme, no Inspector, que o Material Override aponta para esse recurso, e não para um material embutido na Scene.
3. Rode a Scene com F6 e confirme que a textura aparece corretamente no Viewport de jogo, não apenas no editor.

Problemas comuns

- Material criado como recurso embutido (sem salvar como `.tres`), impedindo reutilização em outras Scenes: sempre usar **Save As** para tornar o material um recurso externo.
- Textura do Kenney Prototype Kit não aparece após atribuída: verificar se o arquivo de textura foi de fato importado (aparece no FileSystem Dock) antes de tentar atribuí-lo.
- Textura esticada ou repetida de forma exagerada: ajustar os parâmetros de **UV1 Scale** no `StandardMaterial3D`, comparando visualmente até a proporção ficar coerente com o tamanho do `Chao`.

Boas práticas

- Sempre salvar materiais como recursos externos (`.tres`) na pasta `materials/`, nunca embutidos na Scene — isso permite reaproveitar o mesmo material em múltiplas superfícies, reforçando o princípio de reutilização do `PROJECT_ARCHITECTURE.md` (seção 1).
- Nomear o arquivo do material pela função que ele representa (`chao_prototype.tres`), não por características visuais isoladas (`material_cinza.tres`).

- Evitar valores extremos de Roughness ou Metallic no graybox — o objetivo desta etapa é legibilidade do nível, não acabamento visual final.

Comparação com Unity

Na Unity, o mesmo problema é resolvido por materiais baseados no Standard Shader (ou nos shaders do URP/HDRP), com parâmetros equivalentes (Albedo, Smoothness, Metallic) expostos diretamente no Inspector — uma experiência muito próxima ao StandardMaterial3D do Godot. Quando a Unity precisa de um material customizado além do shader padrão, ela oferece o **Shader Graph**, com filosofia nodal equivalente ao Visual Shader do Godot. A diferença arquitetural relevante aparece no próximo passo: terreno.

Parte 2 — Terrain3D como ferramenta de terreno sem equivalente nativo

Objetivo

Entender por que o Godot depende de um addon de terceiros para modelar terreno, antes de esculpir qualquer coisa no editor.

Conceito

Modelar grandes extensões de terreno manualmente, polígono a polígono, não é viável para nenhuma equipe. Por isso, a maioria das engines 3D modernas oferece uma ferramenta dedicada de escultura de terreno — com pincéis de altura, textura por camadas e otimização própria de malha. O Godot, no entanto, não possui essa ferramenta no núcleo da engine: a solução adotada por esta disciplina é o addon **Terrain3D**, que adiciona ao editor um fluxo de escultura de terreno equivalente ao de outras engines, mas como uma peça externa, não nativa.

Essa ausência não é um detalhe menor — é uma diferença arquitetural real entre o Godot e engines com Terrain nativo, e deve ficar explícita: o mesmo problema (modelar terreno) é resolvido de formas estruturalmente diferentes dependendo da engine escolhida.

Passo a passo

1. Com o addon Terrain3D já habilitado (ver seção "Projeto esperado"), no painel Scene clique com o botão direito sobre o Node raiz `NívelTeste` e selecione **Add Child Node**.
2. Busque e adicione um Node do tipo **Terrain3D**, renomeando-o, se necessário, para manter o padrão de nomenclatura do projeto.
3. No Inspector, com o Node `Terrain3D` selecionado, configure uma nova **Terrain3D Storage** (recurso de dados do terreno), salvando-a em `resources/terrain/` (crie a pasta, caso não exista).
4. Abra o painel de ferramentas do Terrain3D (geralmente exibido na parte inferior do editor quando o Node está selecionado).
5. Selecione a ferramenta de escultura de altura (**Sculpt** ou equivalente) e ajuste o tamanho e a força do pincel para um valor pequeno, adequado a ajustes suaves.
6. No Viewport, esculpa uma elevação suave e uma depressão rasa em áreas distintas do terreno, simulando uma variação básica de relevo.
7. Selecione a ferramenta de textura do Terrain3D e associe pelo menos uma textura do Kenney Prototype Kit a uma camada do terreno.
8. Pinte a textura sobre a área esculpida, cobrindo toda a extensão visível do terreno.
9. Posicione o `Player` (existente desde a Semana 1) sobre uma área plana do terreno esculpido, evitando spawná-lo dentro de uma elevação.
10. Salve a Scene (**Ctrl+S**) e pressione **Play Scene** (F6) para confirmar que o Player aparece corretamente sobre o novo terreno, sem afundar nem flutuar.

Resultado esperado

Um Node `Terrain3D` existe dentro de `level_exploration.tscn`, com uma Terrain3D Storage salva em `resources/terrain/`, contendo uma elevação e uma depressão esculpidas, e ao menos uma textura do Kenney Prototype Kit pintada sobre a superfície. O `Player` está posicionado sobre uma área plana do novo terreno.

Verificando

1. Confirme, na árvore de cena, que o Node `Terrain3D` é filho de `NívelTeste`.
2. Rode a Scene com F6 e mova o Player sobre a área esculpida, confirmando que ele reage corretamente ao relevo (sem atravessar o terreno).
3. Verifique, no painel Output, que nenhum erro relacionado à Terrain3D Storage aparece ao abrir ou rodar a Scene.

Problemas comuns

- Terrain3D Storage não salva como recurso externo, gerando erro ao reabrir o projeto: sempre confirmar que o recurso foi salvo em `resources/terrain/` antes de fechar a Scene.
- Player spawna dentro do relevo esculpido, ficando preso ou lançado para fora do nível: reposicionar o Player manualmente após concluir a escultura, nunca antes.
- Textura do terreno não aparece após pintada: confirmar que a camada de textura foi corretamente associada ao material do Terrain3D antes de tentar pintar sobre ela.
- Confundir a escultura de terreno com a movimentação normal de Nodes na Scene: reforçar que o Terrain3D opera com ferramentas de pincel próprias, ativadas apenas quando o Node `Terrain3D` está selecionado.

Boas práticas

- Esculpir variações suaves de relevo no graybox, evitando elevações abruptas que dificultem a leitura do nível pelo Player nesta etapa inicial.
- Manter a Terrain3D Storage como recurso externo em `resources/terrain/`, seguindo a mesma lógica de separação de dados aplicada aos materiais.
- Testar a movimentação do Player sobre o terreno esculpido antes de considerar a etapa concluída — relevo esculpido sem teste de colisão é uma fonte comum de bugs posteriores.

Comparação com Unity

A Unity possui um sistema de **Terrain nativo**, integrado ao editor desde suas primeiras versões, com ferramentas de escultura, pintura de textura e vegetação embutidas no núcleo da engine — sem depender de nenhum pacote de terceiros para o fluxo básico. O Godot, em contraste, não resolve esse problema nativamente: a disciplina depende do addon Terrain3D para obter um fluxo equivalente. Essa diferença deve ficar explícita para a turma como uma limitação real do Godot frente a engines com Terrain nativo, não apenas uma escolha de ferramenta entre várias equivalentes.

Ao final da semana

Este tutorial cobre apenas o Encontro 1. Ao final da Semana 3 completa (Encontros 1 e 2), o nível de teste deverá possuir terreno e material (este encontro) e iluminação global ativa, além do primeiro build exportado (Encontro 2). Segundo o `PROJECT_ARCHITECTURE.md` (seção 6, Módulo 1), este encontro corresponde à conclusão do item "Cena de teste (graybox)", que depende do item "Player (locomoção)" já concluído na Semana 2 e precede diretamente "Renderização moderna e build".

Desafio

Ajuste ao menos um parâmetro do material aplicado ao `Chao` — cor do Albedo, valor de Roughness ou a textura escolhida — de forma diferente da demonstração, justificando brevemente a escolha em relação à identidade visual pretendida para o próprio nível. Não há solução única.

Checklist

- ☐ Scene `level_exploration.tscn` reaberta sem erros
- ☐ Kenney Prototype Kit importado em `assets/prototype/`
- ☐ StandardMaterial3D criado, salvo em `materials/chao_prototype.tres` e aplicado ao `Chao`
- ☐ Node `Terrain3D` adicionado como filho de `NivelTeste`, com Terrain3D Storage salva em `resources/terrain/`
- ☐ Terreno esculpido com ao menos uma elevação e uma depressão, e textura pintada sobre a superfície
- ☐ Player reposicionado sobre área plana do terreno, testado com F6 sem afundar ou flutuar
- ☐ Desafio de ajuste de material realizado e justificado

Glossário

- **Material Graph:** representação nodal de um shader, que combina texturas, cores e parâmetros sem exigir código de shader direto.
- **StandardMaterial3D:** material pronto do Godot, com parâmetros expostos diretamente no Inspector, usado neste tutorial.
- **Visual Shader:** grafo nodal completo do Godot, equivalente conceitual ao Material Graph, usado quando o StandardMaterial3D não é suficiente (não utilizado neste tutorial).
- **Terrain3D:** addon de terceiros que adiciona ao Godot um fluxo de escultura e texturização de terreno, sem equivalente nativo na engine.
- **Terrain3D Storage:** recurso de dados que armazena a geometria e as texturas do terreno esculpido pelo addon Terrain3D.

Referências

- Godot Documentation — Standard Material 3D:
https://docs.godotengine.org/en/stable/tutorials/3d/standard_material_3d.html

- Terrain3D (addon) — Repositório oficial: <https://github.com/TokisanGames/Terrain3D>
- Orchestrator — Documentação oficial: <https://orchestrator.cratercrash.space/>
- Unity Manual (consulta comparativa) — Terrain: <https://docs.unity3d.com/Manual/script-Terrain.html>
- Kenney Assets (CC0): <https://kenney.nl/>
- GDQuest: <https://www.gdquest.com/>